

Matrices, Inverses and Factorisation

Historical Note

The word *matrix* was coined by James Joseph Sylvester in 1850, who used it for the rectangular array from which determinants could be cut out, in the sense of a womb or mould. It was his friend Arthur Cayley who, in *A Memoir on the Theory of Matrices* of 1858, first treated the array as an object in its own right, with an algebra of its own. Cayley's decisive move was to define the product not entrywise, which would have been the obvious thing, but by the strange-looking rule of this chapter. He did it because a matrix, for him, encoded a substitution of variables, and he wanted the product of two matrices to encode doing one substitution after the other. Everything that follows, including the fact that $AB \neq BA$, is a consequence of that single design decision. In the same memoir Cayley stated what we now call the Cayley–Hamilton theorem, verified it by hand for 2×2 and 3×3 matrices, and remarked that he had not thought it necessary to prove the general case.

Chapter 3 used matrices as bookkeeping: a place to keep the coefficients while we eliminated. That undersells them. A matrix is a linear map written down in coordinates, and once we give matrices an algebra of their own, the whole of elimination turns out to be a statement about multiplying them. This chapter builds that algebra, uses it to settle the uniqueness left open in Theorem 3.9, and ends with the factorisation that every numerical library actually computes when you ask it to solve $Ax = b$.

4.1 The algebra of matrices

Definition 4.1 (Matrices, addition, scaling). An $m \times n$ matrix over $\mathbb{R}$ is a rectangular array $A = (a_{ij})$ with $1 \leq i \leq m$ and $1 \leq j \leq n$; the first index is the row and the second the column. The set of all of them is written $M_{m \times n}(\mathbb{R})$, and $M_n(\mathbb{R})$ when $m = n$. For matrices of the same shape and $\lambda \in \mathbb{R}$,

$$(A + B)_{ij} = a_{ij} + b_{ij}, \quad (\lambda A)_{ij} = \lambda a_{ij}.$$

Both operations act entry by entry, exactly as in Definition 2.2, so an $m \times n$ matrix is a vector of $\mathbb{R}^{mn}$ with its entries arranged in a rectangle rather than a line. Nothing new has happened yet. What makes matrices more than rearranged lists is the product.

4.2 Multiplication, and why it is defined that way

Definition 4.2 (Matrix product). If A is $m \times k$ and B is $k \times n$, their product AB is the $m \times n$ matrix

$$(AB)_{ij} = \sum_{s=1}^k a_{is} b_{sj}.$$

The product is defined only when the number of columns of A equals the number of rows of B .

The entry in position (i, j) is the dot product of row i of A with column j of B , in the sense of Definition 2.8. It is worth drawing once. Figure 4.1 is that drawing: B sits above the product and A to its left, so the row and the column feeding a given entry meet at it.

The definition looks arbitrary until you ask what a matrix is for. In Chapter 3 we read $Ax = b$ as a map applied to a vector, and Definition 4.2 with $n = 1$ says exactly that: $(Ax)_i = \sum_s a_{is} x_s$, the i th equation of the system. Now apply two maps in succession.

Proposition 4.3 (The product is composition). For A of size $m \times k$, B of size $k \times n$ and any $x \in \mathbb{R}^n$,

$$A(Bx) = (AB)x.$$

That is, the matrix AB is the map “do B , then do A ”.

Proof. Both sides are vectors in $\mathbb{R}^m$; compare the i th entry:

$$(A(Bx))_i = \sum_{s=1}^k a_{is} (Bx)_s = \sum_{s=1}^k a_{is} \sum_{r=1}^n b_{sr} x_r = \sum_{r=1}^n \left(\sum_{s=1}^k a_{is} b_{sr} \right) x_r = \sum_{r=1}^n (AB)_{ir} x_r = ((AB)x)_i.$$

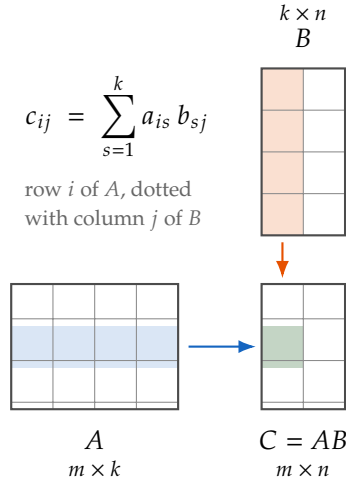
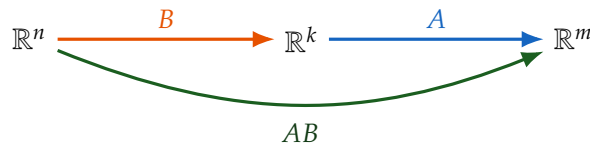


Figure 4.1. Why the shapes must match. Placing B above the product and A to its left makes the rule visible: the shaded entry of C is fed by the shaded row of A and the shaded column of B , and they have the same length k precisely because A has as many columns as B has rows.

Exchanging the two finite sums is legitimate, and it is the only step in the calculation. $\square$



B runs first and A second, so the symbol AB is read right to left.
The order of the letters is the order in which they meet x , reversed.

Figure 4.2. Proposition 4.3 in one picture. Cayley defined the product so that this diagram would commute; every other property of matrix multiplication, including the failure of commutativity, follows from insisting on it.

Key Idea

Matrix multiplication is not a rule to memorise. It is the unique rule that makes matrices compose like the maps they represent. Whenever you cannot remember which index runs over what, recover the definition by asking what $A(Bx)$ has to be.

There is a second reading of the same definition that is often more useful in practice.

Proposition 4.4 (Column and row views). Write $B_1, \dots, B_n$ for the columns of B . Then the columns of AB are $AB_1, \dots, AB_n$. Dually, each row of AB is the corresponding row of A acting on B : row i of AB equals $\sum_s a_{is} \cdot (\text{row } s \text{ of } B)$.

Proof. Column j of AB has entries $(AB)_{ij} = \sum_s a_{is} b_{sj} = (AB_j)_i$, which is the first claim. Row i of AB has entries $\sum_s a_{is} b_{sj}$ for $j = 1, \dots, n$, and that is precisely the combination of the rows of B with weights $a_{i1}, \dots, a_{in}$. $\square$

Key Idea

Left multiplication combines *rows*; right multiplication combines *columns*. Every row operation of Chapter 3 will therefore be a multiplication on the left, and this one sentence is the bridge between the two chapters.

Example 4.5 (A product, three ways). Let

$$A = \begin{pmatrix} 1 & 2 \\ 0 & -1 \\ 3 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 2 & 0 & 1 \\ 1 & 4 & -2 \end{pmatrix}, \quad \text{so } AB \text{ is } 3 \times 3.$$

Entry by entry, $(AB)_{11} = 1 \cdot 2 + 2 \cdot 1 = 4$ and $(AB)_{23} = 0 \cdot 1 + (-1)(-2) = 2$, and continuing,

$$AB = \begin{pmatrix} 4 & 8 & -3 \\ -1 & -4 & 2 \\ 7 & 4 & 1 \end{pmatrix}.$$

By columns: the first column of AB is $A \begin{pmatrix} 2 \\ 1 \end{pmatrix} = 2(1, 0, 3) + 1(2, -1, 1) = (4, -1, 7)$, which is the first column above. By rows: the first row of AB is $1 \cdot (2, 0, 1) + 2 \cdot (1, 4, -2) = (4, 8, -3)$. Three routes, one answer. Use whichever makes the structure of your particular problem visible.

4.3 The rules that hold, and the rules that fail

Theorem 4.6 (Algebraic rules). Whenever the shapes allow the products to be formed,

$$(AB)C = A(BC), \quad \lambda(AB) = (\lambda A)B = A(\lambda B),$$

$$A(B + C) = AB + AC, \quad (A + B)C = AC + BC.$$

Moreover $\mathbf{1}_m A = A = A \mathbf{1}_n$ for the identity matrix $\mathbf{1}_n$, the $n \times n$ matrix with ones on the diagonal and zeros elsewhere.

Proof. For associativity, compare entries:

$$\begin{aligned} ((AB)C)_{ij} &= \sum_s (AB)_{is} c_{sj} = \sum_s \sum_r a_{ir} b_{rs} c_{sj}, \\ (A(BC))_{ij} &= \sum_r a_{ir} (BC)_{rj} = \sum_r \sum_s a_{ir} b_{rs} c_{sj}, \end{aligned}$$

and the two double sums differ only in the order of summation. Distributivity and the scalar rule follow the same way, one line each. For the identity, $(\mathbf{1}_m A)_{ij} = \sum_s \delta_{is} a_{sj} = a_{ij}$, where δ_{is} is 1 if $i = s$ and 0 otherwise. $\square$

We write $\mathbf{1}$ for the identity, following our baseline text; many books write I or I_n , and nothing depends on the choice. Associativity deserves a moment's respect: it is what allows us to write ABC without brackets, and it is the reason a chain of transformations can be collapsed into a single matrix once and reused.

Caution

Two familiar rules of ordinary arithmetic are false for matrices.

Multiplication does not commute. Let $A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$, a shear, and $B = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$, a quarter turn. Then

$$AB = \begin{pmatrix} 1 & -1 \\ 1 & 0 \end{pmatrix}, \quad BA = \begin{pmatrix} 0 & -1 \\ 1 & 1 \end{pmatrix},$$

which are different. They had to be: shearing a rotated picture is not the same as rotating a sheared one, and Figure 4.2 says the algebra must record that.

A product of nonzero matrices can be zero. With $N = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ we have $N \neq 0$ but $N^2 = 0$. So there is no cancellation law: $AB = AC$ does not give $B = C$, and $A^2 = 0$ does not give $A = 0$. Everything you know about solving scalar equations by dividing has to be re-earned, and Section 4.6 is where we earn it.

Insight

Because $A(Bx) = (AB)x$, a stack of linear layers is a single linear layer. A network computing $x \mapsto W_3 W_2 W_1 x$ has exactly the expressive power of the one matrix $W_3 W_2 W_1$, no matter how many layers you pile up. That one observation is the entire reason neural networks interleave a nonlinearity between the matrices: without it, depth buys nothing at all. Associativity, usually filed as a dull axiom, is

the theorem doing the work.

Proposition 4.7 (Cost of a product). *Multiplying two $n \times n$ matrices by Definition 4.2 takes n^3 multiplications and $n^3 - n^2$ additions, so about $2n^3$ arithmetic operations.*

Proof. There are n^2 entries, and each is a sum of n products, requiring n multiplications and $n - 1$ additions. $\square$

Insight

The exponent 3 is not forced. Strassen showed in 1969 that seven multiplications suffice to combine 2×2 blocks where the obvious method needs eight, and recursing gives $O(n^{\log_2 7}) = O(n^{2.807})$. A long line of increasingly intricate work has pushed the theoretical exponent down to roughly 2.37, and nobody knows whether 2 is attainable. Almost none of those algorithms is used: their hidden constants are enormous, and on real hardware a cache-friendly $2n^3$ implementation wins until n is very large. Strassen's method is the one exception that occasionally pays. The lesson generalises, and we will meet it again with iterative solvers: the operation count is only half of the cost model.

4.4 Transpose, and the special families

Definition 4.8 (Transpose). *The transpose of an $m \times n$ matrix A is the $n \times m$ matrix A^t with $(A^t)_{ij} = a_{ji}$: rows become columns. A square matrix is symmetric if $A^t = A$ and antisymmetric if $A^t = -A$.*

Proposition 4.9 (Transpose reverses products). $(A + B)^t = A^t + B^t$, $(\lambda A)^t = \lambda A^t$, $(A^t)^t = A$, and

$$(AB)^t = B^t A^t.$$

Proof. Only the last needs work. Both sides are $n \times m$, and

$$((AB)^t)_{ij} = (AB)_{ji} = \sum_s a_{js} b_{si} = \sum_s (B^t)_{is} (A^t)_{sj} = (B^t A^t)_{ij}. \quad \square$$

The reversal is not a quirk of the algebra. Transposing is the matrix shadow of running a map backwards, and to undo “ B then A ” you must undo A first. The same reversal

appears in Proposition 4.15 below, and for the same reason.

A few families of matrices will recur so often that they are worth naming now. A matrix is *diagonal* if $a_{ij} = 0$ whenever $i \neq j$; *upper triangular* if $a_{ij} = 0$ whenever $i > j$, so that all the nonzero entries sit on or above the diagonal; and *lower triangular* if $a_{ij} = 0$ whenever $i < j$. A triangular matrix with every diagonal entry equal to 1 is called *unit triangular*.

Example 4.10 (Triangularity survives multiplication). *If A and B are both upper triangular and $i > j$, then $(AB)_{ij} = \sum_s a_{is}b_{sj}$, and each term vanishes: if $s < i$ then $a_{is} = 0$, while if $s \geq i > j$ then $b_{sj} = 0$. So AB is upper triangular too, and the same argument works below the diagonal. Notice also that $(AB)_{ii} = a_{ii}b_{ii}$, so the diagonals simply multiply. In particular a product of unit lower triangular matrices is unit lower triangular, which is the fact that makes Section 4.8 work.*

4.5 Elementary matrices: elimination as multiplication

We now cash in the observation that left multiplication combines rows.

Definition 4.11 (Elementary matrices). *For each elementary row operation of Definition 3.3 on m rows, the corresponding elementary matrix is the $m \times m$ matrix obtained by performing that operation on $\mathbf{1}_m$. Explicitly, writing e_i for the i th standard basis column,*

- *swapping rows i and j gives P_{ij} , the identity with those two rows exchanged;*
- *scaling row i by $\lambda \neq 0$ gives $D_i(\lambda)$, the identity with λ in position (i, i) ;*
- *adding μ times row i to row j , with $i \neq j$, gives $E_{ji}(\mu) = \mathbf{1}_m + \mu e_j e_i^t$.*

Proposition 4.12 (Row operations are left multiplications). *Let E be the elementary matrix of a given row operation. Then for every $m \times n$ matrix A , the matrix EA is what that operation produces when applied to A . Moreover every elementary matrix is invertible, and its inverse is elementary of the same kind:*

$$P_{ij}^{-1} = P_{ij}, \quad D_i(\lambda)^{-1} = D_i(1/\lambda), \quad E_{ji}(\mu)^{-1} = E_{ji}(-\mu).$$

Proof. By Proposition 4.4, row r of EA is $\sum_s E_{rs} \cdot (\text{row } s \text{ of } A)$. Reading off the rows of E in each of the three cases gives exactly the described operation: for P_{ij} row i of E is e_j^t , so row i of EA is row j of A ; for $D_i(\lambda)$ row i is λe_i^t ; for $E_{ji}(\mu)$ row j is $e_j^t + \mu e_i^t$.

For the inverses, swapping twice is the identity, and $D_i(\lambda)D_i(1/\lambda) = \mathbf{1}$ entry by entry.

For the third, use $e_i^t e_j = 0$ when $i \neq j$:

$$(1 + \mu e_j e_i^t)(1 - \mu e_j e_i^t) = 1 - \mu^2 e_j (e_i^t e_j) e_i^t = 1. \quad \square$$

Key Idea

Running Gaussian elimination on A is the same as forming $E_N \cdots E_2 E_1 A$ for a sequence of elementary matrices. Since each E_t is invertible, so is the product, and this single sentence proves Proposition 3.4 again in one line: $Ax = b$ and $(EA)x = Eb$ have the same solutions because E can be undone.

Example 4.13 (Elimination, recorded as matrices). *Take the first step of Example 3.7, which was $R_2 \rightarrow R_2 - 2R_1$ on a 3-row matrix. Its elementary matrix is*

$$E_{21}(-2) = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad \text{and} \quad E_{21}(-2) \begin{pmatrix} 1 & 1 & 1 \\ 2 & -1 & 3 \\ -1 & 4 & -2 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & -3 & 1 \\ -1 & 4 & -2 \end{pmatrix},$$

which is what the row operation did. The second step, $R_3 \rightarrow R_3 + R_1$, is $E_{31}(1)$, and the third, $R_3 \rightarrow R_3 + \frac{5}{3}R_2$, is $E_{32}(\frac{5}{3})$. The whole elimination is the single matrix $E_{32}(\frac{5}{3})E_{31}(1)E_{21}(-2)$, and Example 4.10 tells us in advance that it will be unit lower triangular.

4.6 Inverses

Definition 4.14 (Invertible matrix). *A square matrix $A \in M_n(\mathbb{R})$ is invertible if there is a matrix B with $AB = BA = \mathbf{1}_n$. Such a B is unique, for if $AB = \mathbf{1} = CA$ then $C = C(AB) = (CA)B = B$, and we write $B = A^{-1}$.*

Proposition 4.15 (Rules for inverses). *If A and B are invertible of the same size, then so are A^{-1} , AB and A^t , with*

$$(A^{-1})^{-1} = A, \quad (AB)^{-1} = B^{-1}A^{-1}, \quad (A^t)^{-1} = (A^{-1})^t.$$

Proof. The first is immediate from the symmetry of the definition. For the second, $(AB)(B^{-1}A^{-1}) = A(BB^{-1})A^{-1} = AA^{-1} = \mathbf{1}$, and the other order is the same calculation. For the third, transpose $AA^{-1} = \mathbf{1}$ using Proposition 4.9 to get $(A^{-1})^t A^t = \mathbf{1}$. $\square$

Caution

The order reverses. Writing $(AB)^{-1} = A^{-1}B^{-1}$ is the single most common algebraic error in this subject, and it is wrong for the same homely reason that you take off your shoes before your socks.

Which matrices are invertible? Chapter 3 already knows the answer; we only have to translate it.

Theorem 4.16 (The invertibility criterion). *For $A \in M_n(\mathbb{R})$ the following are equivalent.*

- (a) A is invertible.
- (b) $Ax = 0$ has only the solution $x = 0$.
- (c) A has n pivots.
- (d) The reduced row echelon form of A is $\mathbf{1}_n$.
- (e) A is a product of elementary matrices.
- (f) $Ax = b$ has a solution for every $b \in \mathbb{R}^n$, and it is unique.

Proof. (a) $\Rightarrow$ (b): if $Ax = 0$ then $x = \mathbf{1}x = A^{-1}Ax = A^{-1}0 = 0$.

(b) $\Rightarrow$ (c): if A had $r < n$ pivots then by Theorem 3.10(c) the homogeneous system would have $n - r \geq 1$ free parameters and hence a nonzero solution.

(c) $\Rightarrow$ (d): with n pivots in an $n \times n$ matrix every row and every column carries one, so after scaling each pivot to 1 and clearing above it, the reduced form has a single 1 in each row and column in the pattern of Definition 3.6, which is $\mathbf{1}_n$.

(d) $\Rightarrow$ (e): by Proposition 4.12 the reduction is $E_N \cdots E_1 A = \mathbf{1}_n$, so $A = E_1^{-1} \cdots E_N^{-1}$, a product of elementary matrices.

(e) $\Rightarrow$ (a): each elementary matrix is invertible and, by Proposition 4.15, so is any product of them.

Finally (a) $\Rightarrow$ (f) since $x = A^{-1}b$ works and is forced, and (f) $\Rightarrow$ (b) by taking $b = 0$ and using uniqueness. $\square$

Proposition 4.17 (One-sided inverses suffice, for square matrices). *If $A, B \in M_n(\mathbb{R})$ satisfy $BA = \mathbf{1}_n$, then A is invertible and $B = A^{-1}$.*

Proof. If $Ax = 0$ then $x = BAx = 0$, so condition (b) of Theorem 4.16 holds and A is invertible. Then $B = B(AA^{-1}) = (BA)A^{-1} = A^{-1}$. $\square$

Caution

Proposition 4.17 uses squareness twice and fails without it. The 1×2 matrix $A = \begin{pmatrix} 1 & 0 \end{pmatrix}$ and the 2×1 matrix $B = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ satisfy $AB = \mathbf{1}_1$, yet $BA = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \neq \mathbf{1}_2$. In infinite dimensions the failure is worse still: the shift operator on infinite sequences has a left inverse and no right inverse, which is one of the reasons Chapter 6 keeps careful track of finiteness.

Computing the inverse

Theorem 4.16 contains an algorithm. If $E_N \cdots E_1 A = \mathbf{1}$, then $A^{-1} = E_N \cdots E_1 = E_N \cdots E_1 \mathbf{1}$: the very same operations that reduce A to the identity will build A^{-1} out of the identity. So run them side by side, on the augmented array $[A \mid \mathbf{1}]$.

Example 4.18 (Gauss–Jordan inversion). Invert $A = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{pmatrix}$. Start from $[A \mid \mathbf{1}]$ and clear the first column with $R_3 \rightarrow R_3 - 5R_1$:

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & 4 & 0 & 1 & 0 \\ 5 & 6 & 0 & 0 & 0 & 1 \end{array} \right] \longrightarrow \left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & 4 & 0 & 1 & 0 \\ 0 & -4 & -15 & -5 & 0 & 1 \end{array} \right].$$

Now $R_3 \rightarrow R_3 + 4R_2$ finishes the forward sweep, leaving a pivot 1 in the corner:

$$\longrightarrow \left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & 4 & 0 & 1 & 0 \\ 0 & 0 & 1 & -5 & 4 & 1 \end{array} \right].$$

Clear upward with $R_2 \rightarrow R_2 - 4R_3$, then $R_1 \rightarrow R_1 - 3R_3$, then $R_1 \rightarrow R_1 - 2R_2$:

$$\longrightarrow \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & -24 & 18 & 5 \\ 0 & 1 & 0 & 20 & -15 & -4 \\ 0 & 0 & 1 & -5 & 4 & 1 \end{array} \right].$$

The left block is $\mathbf{1}$, so A is invertible and the right block is A^{-1} . Verify one row of AA^{-1} before trusting it: row 1 of A against column 1 of A^{-1} gives $1(-24) + 2(20) + 3(-5) = -24 + 40 - 15 = 1$, and against column 2 gives $1(18) + 2(-15) + 3(4) = 0$, as required.

Caution

Inverting is a fine way to *understand* a matrix and a poor way to *use* one. Computing A^{-1} costs about $2n^3$ operations and then every solve costs a further $2n^2$, whereas the factorisation of the next section costs $\frac{2}{3}n^3$ once and $2n^2$ per solve: cheaper on the first system and no worse on any later one. It is also more accurate, because forming A^{-1} explicitly discards information about how the elimination went. The rule in numerical work is never to write $x = A^{-1}b$ as an instruction to a computer, only as a statement about mathematics.

4.7 Why the reduced form is unique

We can now pay the debt from Theorem 3.9. The proof rests on a single observation: row operations change the entries of a matrix but not the linear relations among its columns.

Lemma 4.19 (Row equivalence preserves column relations). *Let M be invertible and $R = MA$. Then A and R have the same null space, that is, $Ax = 0 \iff Rx = 0$. Consequently, for any coefficients $c_1, \dots, c_{j-1}$, column j of A equals $\sum_{s < j} c_s$ (column s of A) if and only if the same identity holds for the columns of R .*

Proof. If $Ax = 0$ then $Rx = MAX = 0$; if $Rx = 0$ then $Ax = M^{-1}Rx = 0$. For the second statement, the relation $A_j = \sum_{s < j} c_s A_s$ says exactly that $Ax = 0$ for the vector x with entries $c_1, \dots, c_{j-1}$ in the first $j-1$ places, -1 in place j , and zeros after. Since the null spaces agree, that vector kills A if and only if it kills R . $\square$

Lemma 4.20 (Pivot columns are the independent ones). *Let R be in reduced row echelon form with pivots in columns $j_1 < \dots < j_r$. Then column j of R is a linear combination of the columns strictly to its left precisely when j is not a pivot column, and in that case the coefficients are the entries of column j itself.*

Proof. In reduced form the pivot column j_i is the standard basis vector e_i , since its pivot is 1 and every other entry of that column is 0. Suppose j is a free column and let i be the number of pivots strictly to the left of j . By the echelon shape every entry of column j below row i is zero, so

$$R_j = \sum_{s=1}^i R_{sj} e_s = \sum_{s=1}^i R_{sj} R_{j_s},$$

a combination of the pivot columns to its left with the stated coefficients. Conversely any pivot column $R_{j_i} = e_i$ cannot be a combination of the columns before it, because every

one of those has a zero in row i : all the earlier pivot columns are $e_1, \dots, e_{i-1}$, and every earlier free column, by what we just proved, is a combination of those. $\square$

Proof of Theorem 3.9. Existence is the algorithm of Chapter 3. For uniqueness, suppose R and S are both reduced row echelon forms of A . By Proposition 4.12 there are invertible M and M' with $R = MA$ and $S = M'A$, so both have the same column relations as A by Lemma 4.19.

By Lemma 4.20, the pivot columns of a reduced matrix are exactly those columns that are *not* combinations of their predecessors. Since R and S share that property column by column, they have pivots in the same positions, say $j_1 < \dots < j_r$. Their pivot columns therefore agree, both being $e_1, \dots, e_r$. And for a free column j , Lemma 4.20 says its entries *are* the coefficients expressing it in terms of the earlier pivot columns; those coefficients are determined by A alone, so that column of R agrees with that column of S as well. Every column agrees, hence $R = S$. $\square$

Key Idea

The number of pivots is now unambiguously a property of the matrix, not of the person doing the elimination. That number is the *rank*, written $\text{rank } A$, and Chapter 6 is largely the study of what it measures.

4.8 The LU factorisation

Elimination does not merely produce an answer; it produces a factorisation, and keeping it is what makes repeated solves cheap.

Suppose we can run forward elimination on a square A using only operations of type (R3), adding multiples of a pivot row to the rows below. By Example 4.13 the result is $E_N \cdots E_1 A = U$ with U upper triangular, and each E_t is unit lower triangular. Hence

$$A = (E_N \cdots E_1)^{-1} U = L U, \quad L = E_1^{-1} \cdots E_N^{-1},$$

and L is unit lower triangular by Example 4.10. There is a pleasant surprise in the bookkeeping: because the operations are applied column by column, no cancellation occurs between them, and L is simply the table of multipliers used.

Theorem 4.21 (LU factorisation). *If Gaussian elimination on $A \in M_n(\mathbb{R})$ can be completed without row swaps, then $A = LU$ where U is the resulting upper triangular matrix and L is unit lower triangular with L_{ji} = the multiplier used to eliminate the entry in position (j, i) . If row swaps are required, then instead $PA = LU$ for a permutation matrix P recording those swaps.*

Proof. For the swap-free case the factorisation was derived above; what remains is to see that L is the table of multipliers. Eliminating the entry in position (j, i) subtracts L_{ji} times row i from row j , so $E_t = \mathbf{1} - L_{j_i i_t} e_{j_i} e_{i_t}^t$ and, by Proposition 4.12, $E_t^{-1} = \mathbf{1} + L_{j_i i_t} e_{j_i} e_{i_t}^t$. In the product $L = E_1^{-1} \cdots E_N^{-1}$ every cross term carries a factor $e_{i_t}^t e_{j_{t'}}^{t'}$ with $t < t'$, and that vanishes because elimination works left to right: $j_{t'} > i_{t'} \geq i_t$, so the two indices can never agree. Hence $L = \mathbf{1} + \sum_t L_{j_i i_t} e_{j_i} e_{i_t}^t$, which is the claim.

With swaps, elimination produces $E_s P_s \cdots E_1 P_1 A = U$, where P_k exchanges two rows numbered k or higher and E_k clears column k . A transposition P of two rows $a, b > i$ fixes row i , so $e_i^t P = e_i^t$ and $P e_j = e_{P(j)}$, giving

$$P(\mathbf{1} + \mu e_j e_i^t) = P + \mu e_{P(j)} e_i^t = (\mathbf{1} + \mu e_{P(j)} e_i^t)P.$$

Every swap can therefore be moved to the right past the elimination factors that precede it, at the cost of permuting the multipliers already stored. Collecting them gives $\tilde{E}_s \cdots \tilde{E}_1 P A = U$ with $P = P_s \cdots P_1$, and the swap-free argument applied to PA finishes it. $\square$

Figure 4.3 shows the shape of that statement: the multipliers thrown away during the forward sweep are precisely what is stored below the diagonal of L , so nothing computed is discarded.

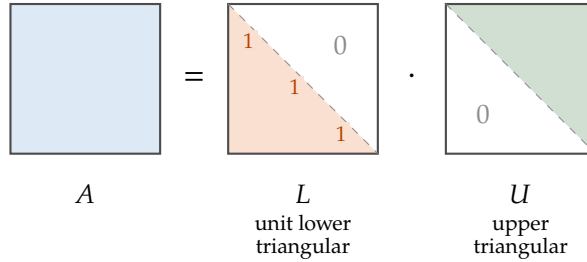


Figure 4.3. The shape of Theorem 4.21. Elimination does not destroy A ; it splits it. The multipliers that were subtracted away are exactly the entries stored below the diagonal of L , and the ones on the diagonal of L are there because a row is never rescaled during the forward sweep.

Example 4.22 (A factorisation, and a solve). *Take*

$$A = \begin{pmatrix} 2 & 1 & 1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{pmatrix}.$$

Eliminate: $R_2 \rightarrow R_2 - 2R_1$ uses multiplier 2, and $R_3 \rightarrow R_3 + R_1$ uses multiplier -1 ,

giving rows $(0, -8, -2)$ and $(0, 8, 3)$. Then $R_3 \rightarrow R_3 + R_2$ uses multiplier -1 and gives $(0, 0, 1)$. Collecting the multipliers into L and the result into U ,

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 2 & 1 & 1 \\ 0 & -8 & -2 \\ 0 & 0 & 1 \end{pmatrix}.$$

Check the last row of LU : $-1(2, 1, 1) - 1(0, -8, -2) + 1(0, 0, 1) = (-2, 7, 2)$, which is row 3 of A .

Now solve $Ax = b$ with $b = (5, -2, 9)$ in two triangular sweeps. First $Ly = b$, downward:

$$y_1 = 5, \quad 2y_1 + y_2 = -2 \implies y_2 = -12,$$

$$-y_1 - y_2 + y_3 = 9 \implies y_3 = 9 + 5 - 12 = 2.$$

Then $Ux = y$, upward:

$$x_3 = 2, \quad -8x_2 - 2x_3 = -12 \implies x_2 = 1, \quad 2x_1 + x_2 + x_3 = 5 \implies x_1 = 1.$$

So $x = (1, 1, 2)$, and $Ax = (2 + 1 + 2, 4 - 6 + 0, -2 + 7 + 4) = (5, -2, 9)$ as required. Neither sweep touched A again: for a second right-hand side we would repeat only the two sweeps, at $O(n^2)$ cost.

Example 4.23 (When a swap is unavoidable). The matrix $A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ has no LU factorisation. If $A = LU$ with L unit lower triangular and U upper triangular then the $(1, 1)$ entry of LU is U_{11} , forcing $U_{11} = 0$; but then the $(2, 1)$ entry is $L_{21}U_{11} = 0 \neq 1$. The cure is a swap: with $P = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ we get $PA = \mathbf{1} = \mathbf{1} \cdot \mathbf{1}$. Since the partial pivoting of Chapter 3 performs swaps anyway, the factorisation a library returns is always the $PA = LU$ form.

A swap can be worth making even when elimination would go through without one. Figure 4.4 runs the same 2×2 system both ways: the matrix is well conditioned, so the answer is not delicate, and yet eliminating on the tiny corner entry loses it entirely while a single swap recovers it.

Key Idea

Solve $Ax = b$ by factoring, not by inverting. Factor once at about $\frac{2}{3}n^3$ operations, then answer each new right-hand side with a forward sweep through L and a

no pivoting. multiplier $1/\varepsilon = 10^{20}$;
 $u_{22} = 1 - 10^{20}$ rounds to -10^{20} ,
 and the 1 is gone. Answer: (0, 1).

$$A = \begin{pmatrix} \varepsilon & 1 \\ 1 & 1 \end{pmatrix}, \quad \varepsilon = 10^{-20}$$

swap the rows first. multiplier ε ;
 nothing is lost to rounding.
 Answer: (1, 1), correct.

the matrix is perfectly
 well conditioned

the problem is fine; only the ordering of the operations was wrong

Figure 4.4. Why elimination pivots. The matrix $\begin{pmatrix} \varepsilon & 1 \\ 1 & 1 \end{pmatrix}$ has condition number close to 1, so the answer is not sensitive to the data at all. Eliminating in the given order nonetheless destroys it: the multiplier is $1/\varepsilon = 10^{20}$, and forming $1 - 10^{20}$ in floating point discards the 1 entirely, after which the computed solution is (0, 1) instead of (1, 1). Swapping the two rows first makes the multiplier ε , and every subtraction is then harmless. This is the same distinction Chapter 17 draws throughout: a well conditioned problem solved by an unstable algorithm, where the fix costs one comparison per column.

backward sweep through U , at about $2n^2$ operations. This is what `lu`, `solve` and their equivalents in every numerical library actually do, and it is why they return a factorisation object rather than a number.

4.9 Blocks

A matrix may be cut into rectangular blocks, and if two partitions are compatible then the product can be computed blockwise, treating each block as if it were a single entry. For

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad AB = \begin{pmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{pmatrix},$$

provided every product written down has matching inner dimensions. The proof is Definition 4.2 with the summation index split into ranges, and the order of the factors within each term must be preserved, since blocks do not commute either.

Insight

Block multiplication is why modern hardware can multiply large matrices at all. Cutting A and B into tiles that fit in cache turns one enormous memory-bound computation into many small compute-bound ones, at no change in the arithmetic. It is also how a batch of inputs is processed in a neural network: stacking N input

vectors as the columns of a matrix X turns N separate computations Wx_i into the single product WX , which is Proposition 4.4 read as an engineering instruction. Strassen's algorithm is block multiplication too, with a cleverer bookkeeping of the eight products into seven.

Open Problem 4.24 (Fast and stable at the same time). *Strassen's algorithm multiplies matrices in fewer than n^3 operations, and by the reductions of this chapter it also inverts and factors them faster. But the classical error analysis of factorisation assumes the classical algorithm. Is there an LU factorisation running in $O(n^{\omega+\varepsilon})$ operations that is backward stable in the componentwise sense, that is, whose computed factors are the exact factors of $A + \Delta A$ with $|\Delta A| \leq c(n) u |L||U|$?*

What is known. *Fast matrix multiplication is not backward stable in the strong sense: it satisfies only a normwise bound, because it forms and cancels large intermediate quantities that never appear in the classical algorithm [321]. Demmel, Dumitriu and Holtz showed that this is enough for a great deal, giving fast algorithms for factorisation and eigenvalue problems that are stable in the normwise sense [183]. The componentwise question, which is what the classical theory delivers and what sparse and structured problems rely on, is open.*

Where the difficulty sits. *Strassen's seven products include differences such as $A_{11} - A_{21}$, and if those two blocks nearly cancel, the relative accuracy of that intermediate is lost and never recovered. The classical algorithm never subtracts two entries of A that the answer does not subtract. So the question is whether a fast algorithm can be arranged to respect the sign structure of the problem, and no known fast scheme does.*

Where a reader might start. *Multiply $A = \begin{pmatrix} 1 & 0 \\ 0 & \varepsilon \end{pmatrix}$ by itself with Strassen's method in arithmetic carrying six digits, at $\varepsilon = 10^{-3}$, and compare with the classical product. The exact $(2, 2)$ entry is $\varepsilon^2 = 10^{-6}$; the classical algorithm returns it and Strassen's returns 0, a relative error of 1. The culprit is $M_1 = (a_{11} + a_{22})(b_{11} + b_{22}) = (1 + \varepsilon)^2 = 1.002001$, which rounds to 1.002: the ε^2 it was carrying is gone before the sum $M_1 - M_2 + M_3 + M_6$ assembles the entry. A quantity of size 10^{-6} was built out of quantities of size 1, which is precisely the difference between a normwise bound and a componentwise one. Chapter 17 sets up the error model this question is asked in, and Chapter 100 gives the algorithm.*

Exercises

Exercise 4.25. With $A = \begin{pmatrix} 2 & -1 \\ 0 & 3 \end{pmatrix}$ and $B = \begin{pmatrix} 1 & 4 \\ 5 & -2 \end{pmatrix}$, compute AB and BA and confirm that they differ. Then compute $(AB)^t$ and $B^t A^t$ and confirm that they agree.

Exercise 4.26. Find a 3×3 matrix N with $N \neq 0$, $N^2 \neq 0$ and $N^3 = 0$. Hint: put the nonzero entries just above the diagonal and think about what left multiplication by N does to the standard basis columns.

Exercise 4.27. Write down the 3×3 elementary matrices for the operations $R_1 \leftrightarrow R_3$, $R_2 \rightarrow 4R_2$ and $R_3 \rightarrow R_3 - 2R_1$, and verify in each case that the inverse is the elementary matrix of the reversed operation.

Exercise 4.28. Use Gauss–Jordan elimination on $[A \mid \mathbf{1}]$ to invert $A = \begin{pmatrix} 1 & 0 & 2 \\ 2 & -1 & 3 \\ 4 & 1 & 8 \end{pmatrix}$, and check your answer by computing one row of AA^{-1} .

Exercise 4.29. Show that $A^t A$ is symmetric for every $m \times n$ matrix A , of whatever shape. Then show that $A + A^t$ is symmetric and $A - A^t$ is antisymmetric for square A , and deduce that every square matrix is the sum of a symmetric and an antisymmetric matrix in exactly one way.

Exercise 4.30. Find the LU factorisation of $A = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 5 & 4 \\ 3 & 8 & 8 \end{pmatrix}$ and use it to solve $Ax = (4, 11, 19)$. Then solve $Ax = (1, 1, 1)$ reusing the same factorisation, and count how many arithmetic operations the second solve required compared with the first.

Exercise 4.31. Prove that an upper triangular matrix is invertible if and only if every diagonal entry is nonzero, and that its inverse is then upper triangular as well. Hint: for the second part, consider what Theorem 4.16 and Gauss–Jordan elimination do to a triangular array.

Exercise 4.32. Give a 2×3 matrix A and a 3×2 matrix B with $AB = \mathbf{1}_2$, and compute BA . Explain why no choice of A and B of these shapes can also give $BA = \mathbf{1}_3$. Hint: if $BA = \mathbf{1}_3$ then $Ax = 0$ forces $x = 0$; now apply Proposition 3.13 to A , which is wide.

Exercise 4.33. Where exactly does the proof in Section 4.7 use the word reduced? Exhibit two different row echelon forms, not reduced, of the same matrix, and confirm that they nonetheless have their pivots in the same columns.

References and further reading

Sylvester's coinage of the word and Cayley's 1858 memoir are the standard landmarks in the prehistory of matrix algebra, and Cayley's own framing of the product as composition of substitutions is the one we have followed here. Our proof that the reduced row echelon form is unique goes through the null space and the column relations, which is the argument we find clearest; Hefferon [307] proves it by a careful induction on the leading entries instead, and comparing the two is instructive. Austin [35] is particularly good on the column view of the product, which is the reading that generalises best to the numerical algorithms of the later chapters. Banica [51], our baseline, develops matrix algebra with an eye to the operator-theoretic material of the later parts, and the notation of this chapter follows his. For the LU factorisation, its pivoted variant, the operation counts and the reasons never to form an explicit inverse, Trefethen and Bau [607] remains the reference, and their treatment of why partial pivoting is stable in practice despite being unstable in the worst case is one of the best short pieces of writing in the subject.